sellingstudio 코드 리뷰

리뷰 일자: 2026-10-11 / 대상 경로: D:₩WorkSpace₩platform₩sellingstudio (android/, ios/는 Capacitor 생성물로 구조만 확인)

1. 프로젝트 개요

- 한 줄 설명: 한국 오픈마켓 셀러를 위한 AI 상세페이지 빌더 SaaS. 텍스트/이미지/URL/음성 입력 -> OpenRouter(Claude 카피, GPT Vision, Gemini 네이밍, DeepSeek 분석) -> fal.ai 이미지 생성(webhook + Realtime) -> 크레딧 차감. Capacitor로 Android/iOS 패키징.
- 목적/대상 사용자: 쿠팡·스마트스토어·와디즈·11번가 셀러. 크레딧 기반 과금(FREE 15 / STARTER 150 / PRO 350 / ENTERPRISE 550).
- 문서 상태: PRD.md(SellingStudio 3.0)와 CLAUDE.md가 비교적 구체적. 단 CLAUDE.md "현재 개발 환경 상태"(9-18행)는 "Supabase 클라이언트 미작성, 페이지 미작성"으로 초기 상태 그대로이고, middleware.ts thin wrapper 패턴을 지시하지만 코드는 Next 16의 proxy.ts 컨벤션으로 구현되어 문서가 뒤처짐.

영역	기술
프론트	Next.js 16.2.7 (App Router), React 19.2, Tailwind v4, shadcn/ui(radix-ui 통합 패키지) 14종, Zustand 5, sonner, @xyflow/react 12, lucide-react
백엔드	Next.js Route Handlers 11개(app/api/), zod 4 입력 검증, proxy.ts(Next 16 미들웨어) 세션 갱신·라우트 보호
DB	Supabase PostgreSQL. supabase/migrations/001(profiles, credit_logs, selling_pages, image_jobs, image_prompts + RLS + Realtime + 가입 트리거), 002(admin 플랜)
인프라	Vercel 가정. Capacitor 8(Android/iOS, camera/filesystem/share/haptics/push/splash/status-bar). CI·테스트 없음(playwright devDependency만 존재)
AI	OpenRouter: deepseek/deepseek-chat-v4-flash(URL 분석), anthropic/claude-sonnet-4-6(카피), openai/gpt-5-mini(Vision), google/gemini-2-5-flash(네이밍). fal.ai: flux schnell/dev/pro/pro-ultra(플랜별), whisper(STT). Browserless(크롤링, 현재 미작동)

- 현재 완성도 추정: 약 55%.
  - 근거: 핵심 경로(로그인 -> 상세페이지 생성 폼 -> 크레딧 차감 -> 카피 생성 -> selling_pages 저장 -> 이미지 잡 큐 -> webhook -> Realtime 뷰어)가 실제 코드로 연결되어 있고 any가 0건, 전 API 라우트에 getUser() 인증과 zod 검증이 있다. 반면 대시보드의 Projects/Billing/Products/Sales/Integrations 탭은 목 데이터(components/dashboard/pages.tsx:55,1107,1940,2442), Stripe 결제·추가 크레딧·월 초기화·엑셀 내보내기·ZIP·플랫폼별 리사이즈(Sharp)는 미구현(패키지만 설치). 네이티브 패키징 설정은 개발 서버를 가리키는 상태로 배포 불가.
- 핵심 기능 목록
  - Google/Kakao OAuth 로그인, 콜백에서 profiles 생성
  - 대시보드 홈(크레딧/플랜 표시) + 8개 탭(대부분 목)
  - 스튜디오 신규 생성(2,919줄): 텍스트/이미지 Vision/URL 벤치마크/음성 STT/템플릿·엑셀 대량 생성, 상품명·키워드 추천
  - 크레딧 차감 API, 잔액 조회 API
  - 이미지 생성 큐(fal.ai) + webhook 수신 + Realtime 카드 업데이트 뷰어
  - 자동화 캔버스(@xyflow v12, 테스트 실행 UI만)
  - Capacitor 네이티브 브리지(카메라/파일/공유/햅틱/푸시) 유틸

2. 구조 분석

경로	역할	비고
proxy.ts, lib/proxy.ts	Next 16 proxy(구 middleware). Supabase 세션 갱신, /dashboard/ 비로그인 리다이렉트	정상. 문서(CLAUDE.md:63-72)만 구식
app/(dashboard)/layout.tsx, dashboard/page.tsx, studio/layout.tsx, studio/new/page.tsx	서버 컴포넌트에서 getUser() + profiles 조회	같은 profiles 조회가 레이아웃->페이지->스튜디오 레이아웃에서 3중 반복
app/api/credits/{use,status}	크레딧 차감(zod, 402) / 잔액	차감이 read-then-write(3장 #2)
app/api/generate/{copy,images,keywords, product-names,analyze,voice}	OpenRouter/fal.ai 호출	크레딧 검증 없음(3장 #1), plan을 바디에서 신뢰(3장 #3)
app/api/webhook/fal/route.ts	fal.ai 완료 콜백 -> image_prompts/image_jobs 갱신(service_role)	서명 검증 없음(3장 #4)
app/api/crawl/url/route.ts	Browserless 또는 직접 fetch -> DeepSeek 분석	env 이름 불일치로 Browserless 항상 미사용, SSRF, 크레딧 기록 불일치(3장 #6)
app/api/auth/callback/route.ts	PKCE 코드 교환 + profiles 보장	next 미검증, DB 트리거와 중복 insert
lib/supabase/{client,server,admin}.ts	SSR 클라이언트 3종	CLAUDE.md 규칙과 일치
lib/llm/{openrouter,prompts,analyze}.ts, lib/image/fal.ts	외부 AI 호출·프롬프트	타임아웃 없음
lib/store/	authStore/creditStore/projectStore	projectStore 미사용, AuthProvider도 어디서도 렌더되지 않음
lib/capacitor/, hooks/, components/capacitor/AppInit.tsx	네이티브 브리지	camera.ts/useNativeCamera는 UI에서 미사용
components/studio/StudioNewForm.tsx	2,919줄 단일 파일: 탭 7종, 대량 생성 루프, 프리셋, 초안 저장	거대 컴포넌트(3장 #8)
components/dashboard/pages.tsx	2,515줄: 8개 탭 페이지 + 자동화 에디터 + 모달	목 데이터 집합(3장 #9)
components/dashboard/workspace.tsx, dashboard-client.tsx	973줄/549줄 클라이언트 셸	pages.tsx가 workspace의 Placeholder를 역참조
capacitor.config.ts	webDir: 'public', server.url: localhost:3000, cleartext, allowMixedContent	배포 불가 설정(3장 #5)
next.config.ts	CSP(unsafe-eval, connect-src https:), allowedDevOrigins에 사설 IP	느슨한 CSP, 개발 IP 하드코딩
image.png, image-1.png(루트, git 추적), public/videos/hero-bg.mp4(2.8MB)	스크린샷·배경 영상	루트 이미지 2개는 정리 대상

데이터 흐름 요약: 서버 컴포넌트가 profiles를 읽어 클라이언트 셸에 넘기고, 스튜디오 폼은 브라우저에서 /api/credits/use로 먼저 크레딧을 차감한 뒤 /api/generate/copy로 카피를 받아 **브라우저가 직접** selling_pages에 insert하고(StudioNewForm.tsx:720-735), 이어서 /api/generate/images로 fal.ai 큐를 등록한다. fal.ai는 /api/webhook/fal로 완료를 알리고 서버가 service_role로 image_prompts를 갱신하면 Supabase Realtime이 StudioViewer에 푸시한다. 즉 "크레딧 차감"과 "생성"이 서로 독립된 요청 두 개로 분리되어 있고, 생성 라우트는 결제 여부를 전혀 알지 못한다.

3. 발견된 문제점

#	심각도	분류	위치	내용
1	높음	보안	app/api/generate/copy/route.ts:17-44, images/route.ts, keywords, product-names, analyze, voice	생성 라우트 6개가 크레딧 잔액·차감을 확인하지 않음. 클라이언트가 /api/credits/use를 건너뛰고 직접 호출하면 무료로 Claude/fal.ai 사용. CLAUDE.md "인증->크레딧 확인->처리" 규칙 위반
2	높음	버그	app/api/credits/use/route.ts:33-61	잔액 SELECT 후 UPDATE하는 비원자 차감. 동시 요청 2건이면 둘 다 402를 통과해 음수 잔액/중복 차감. PRD 9장 "트랜잭션 보장" 미충족. credit_logs insert 실패도 무시
3	높음	보안	app/api/generate/images/route.ts:15,33-34	이미지 모델을 결정하는 plan을 요청 바디에서 받음. free 사용자가 plan:'enterprise'를 보내면 flux-pro-ultra($0.06/장) 사용. 서버의 profiles.plan을 써야 함
4	높음	보안	app/api/webhook/fal/route.ts:13-27, supabase/migrations/001_initial_schema.sql:134-136	webhook에 서명/시크릿 검증이 없어 request_id만 알면 임의 이미지 URL을 completed로 주입 가능. 또 image_prompts_service_role_update 정책이 USING(true) WITH CHECK(true)로 모든 로그인 사용자가 타인 image_prompts를 UPDATE 가능(service_role은 RLS를 우회하므로 이 정책은 불필요하고 유해)
5	높음	설정	capacitor.config.ts:5-15,37-39	webDir: 'public'(빌드 산출물 아님), server.url: 'http://localhost:3000', cleartext: true, allowMixedContent: true. 네이티브 앱이 개발 PC의 dev 서버를 로드하는 구성이라 스토어 배포 불가. API Route를 쓰는 Next.js는 static export가 안 되므로 배포 URL을 가리키는 원격 로딩 전략으로 확정해야 함
6	중간	버그	app/api/crawl/url/route.ts:9 vs .env.local(BROWSERLESS_API_KEY)	코드는 BROWSERLESS_TOKEN을 읽어 항상 미설정 -> Browserless 미사용, 직접 fetch 폴백만 동작. :95-102는 credit_logs에 -2를 기록하지만 profiles 잔액은 차감하지 않고 balance_after:0 기록(장부 불일치)
7	중간	보안	app/api/crawl/url/route.ts:25-33, app/api/auth/callback/route.ts:8,12-17	사용자 URL 직접 fetch(SSRF, 사설 IP 차단 없음). 콜백은 Host 헤더로 origin을 구성하고 next 파라미터를 검증 없이 리다이렉트(오픈 리다이렉트)
8	중간	구조	components/studio/StudioNewForm.tsx(2,919줄)	탭 7개, 대량 생성 루프, 프리셋 관리, 초안 저장, API 호출 14건이 한 파일. useState 60개 이상. 크레딧 차감->카피->DB insert->이미지 큐 오케스트레이션이 두 곳(:700-745, :2017-2092)에 중복
9	중간	품질	components/dashboard/pages.tsx(2,515줄)	Projects/Billing/Products/Sales/Integrations 탭이 MOCK_* 상수로 렌더. 실제 selling_pages·credit_logs 조회 없음. 랜딩용 "완성된 것처럼 보이는" 화면이 운영 코드에 섞임
10	중간	성능	app/(dashboard)/layout.tsx, dashboard/page.tsx:11-15, studio/layout.tsx:13-17, studio/new/page.tsx	요청 1회에 auth.getUser() 3~4회, profiles 조회 2~3회 중복. studio/[id]/page.tsx:26-38은 쿼리 안에 await 쿼리를 중첩
11	중간	구조	components/studio/StudioNewForm.tsx:720-735, components/dashboard/pages.tsx(avatars upload)	클라이언트가 selling_pages에 직접 insert(RLS로 소유자 보장은 되지만 credit_used, used_model을 클라이언트가 임의 기재). 서버 라우트에서 생성 결과와 함께 insert해야 장부 신뢰 가능

#	심각도	분류	위치	내용
12	중간	설정	package.json	미사용 의존성 다수: @anthropic-ai/sdk, openai, stripe, @stripe/stripe-js, @tanstack/react-query, framer-motion, react-dropzone, react-hook-form, @hookform/resolvers, sharp, date-fns, @capacitor/preferences, @capacitor/camera(lib만), playwright. 번들·설치 시간·보안 감사 범위 증가
13	중간	설정	next.config.ts:4-9,27-36	CSP에 'unsafe-eval' 'unsafe-inline', connect-src https: 전체 허용. allowedDevOrigins에 사설 IP 192.168.35.211 하드코딩
14	낮음	품질	components/dashboard/AuthProvider.tsx, lib/store/projectStore.ts, lib/capacitor/camera.ts, hooks/useNativeCamera.ts, lib/image/fal.ts:53-104(getFalResult, submitFalJobSync)	어디서도 import되지 않는 코드
15	낮음	설정	supabase/migrations/002_add_admin_plan.sql:14-24	개인 이메일을 마이그레이션에 하드코딩. 시드/운영 스크립트로 분리 필요
16	낮음	품질	image.png, image-1.png(루트, 동일 크기 40KB, git 추적), tsconfig.tsbuildinfo	스크린샷·빌드 캐시 잔재
17	낮음	버그	app/api/webhook/fal/route.ts:28-38	완료 이미지 URL을 draft_url에만 기록. 플랜별 고화질 "최종" 모델로 생성해도 final_url은 영원히 null. PRD 4.4 시안->최종 2단계 흐름 미구현
18	낮음	품질	lib/llm/openrouter.ts:23, lib/image/fal.ts:35, app/api/generate/voice/route.ts:55,69	외부 호출에 타임아웃 없음(crawl/url만 AbortSignal.timeout). JSON.parse(content)가 실패하면 500으로 일반화

심각도 높음 상세

#1 크레딧 검증 없는 생성 라우트. 클라이언트는 /api/credits/use(StudioNewForm.tsx:700)를 호출한 뒤 별도로 /api/generate/copy(:707)를 호출한다. 두 요청은 서로를 알지 못하며 copy/route.ts는 인증만 확인한 뒤 바로 Claude를 호출한다(:19-40). images, keywords, product-names, analyze, voice도 동일하다. 즉 개발자 도구에서 /api/generate/만 호출하면 크레딧 0으로 모든 AI 기능을 쓸 수 있다. 해법은 각 생성 라우트 안에서 "잔액 확인 -> 차감 -> 생성 -> 실패 시 환불"을 수행하거나, 차감 라우트가 1회용 토큰을 발급하고 생성 라우트가 이를 소비하는 방식이다.

#2 비원자 크레딧 차감. credits/use/route.ts:33-61은 profiles를 읽어 JS에서 계산한 뒤 UPDATE한다. 동시 요청(대량 생성 루프 StudioNewForm.tsx:695-700는 순차지만, 탭 2개나 재시도로 쉽게 병렬 발생)에서 두 요청이 같은 잔액을 읽고 각각 통과한다. Postgres 함수 deduct_credits(user_id, cost) RETURNS balance를 SECURITY DEFINER로 만들어 UPDATE ... SET monthly_credits = ... WHERE id = $1 AND monthly_credits + extra_credits >= $2 RETURNING ... 한 문장으로 처리하고 credit_logs insert를 같은 함수 안에 넣어야 한다.

#3 클라이언트가 결정하는 이미지 모델. images/route.ts:15는 plan을 zod enum으로 받아 IMAGE_MODELS[plan](:34)으로 모델을 고른다. 서버 컴포넌트가 넘긴 plan prop을 클라이언트가 바디에 넣는 구조(StudioNewForm.tsx:1721,2092)이므로 조작이 자유롭다. 라우트에서 profiles.plan을 조회해 모델과 최대 이미지 수(PLAN_IMAGE_COUNT)를 서버에서 결정해야 한다.

#4 webhook 무검증 + 전체 공개 UPDATE 정책. webhook/fal/route.ts는 바디의 request_id로 image_prompts를 찾아 상태와 URL을 덮어쓴다. fal.ai는 webhook 서명 헤더(X-Fal-Webhook-Signature 등)를 제공하므로 검증하거나 최소한 URL에 시크릿 토큰을 포함해야 한다. 더 심각한 것은 001_initial_schema.sql:134-136의 FOR UPDATE USING (true) WITH CHECK (true) 정책이다. service_role은 RLS 자체를 우회하므로 이 정책은 webhook에 불필요하고, 대신 authenticated 역할이 모든 행을 UPDATE할 수 있게 열어 놓는다. 즉 로그인한 어떤

사용자도 다른 사용자의 이미지 URL을 바꿀 수 있다. 정책을 삭제해야 한다.

#5 배포 불가능한 Capacitor 구성. capacitor.config.ts는 webDir: 'public'과 server.url: 'http://localhost:3000'을 사용한다. 이 설정으로 cap sync하면 네이티브 앱은 개발자 PC의 dev 서버를 평문(cleartext: true)으로 로드한다. Next.js API Route·서버 컴포넌트를 쓰는 이 앱은 output: 'export'가 불가능하므로, 네이티브 앱은 "배포된 HTTPS URL을 로드하는 셸"로 확정하고 server.url을 환경별로 분리(CAP_SERVER_URL), cleartext/allowMixedContent를 제거하며, allowNavigation의 openrouter.ai 와일드카드, fal 와일드카드는 서버가 대신 호출하므로 불필요하다. 또는 Capacitor 쪽을 별도 Vite 정적 프론트로 분리하는 결정이 필요하다.

4. 효율성 개선 제안

구조/아키텍처

- "크레딧 -> 생성 -> 저장"을 서버 한 트랜잭션으로: /api/generate/copy가 deduct_credits RPC 호출 -> OpenRouter -> selling_pages insert -> 응답. 클라이언트의 직접 selling_pages insert와 별도 /api/credits/use 호출 제거.
- requireUserWithProfile() 서버 헬퍼로 getUser()+profiles 조회를 1회로 통합하고 React.cache로 요청 내 중복 제거. 레이아웃->페이지 3중 조회 해소.
- StudioNewForm.tsx를 studio/tabs/{TextTab,ImageTab,VoiceTab,UrlBenchmarkTab,TemplateSelectTab,ExcelUploadSection,KeywordPanel}.tsx, studio/useGeneratePipeline.ts(오케스트레이션 훅), studio/constants.ts로 분해. 대량 생성 루프와 단건 생성의 중복 제거.
- pages.tsx의 8개 탭을 파일별로 분리하고, 목 데이터를 실제 selling_pages/credit_logs 조회로 교체 또는 "준비 중" 상태로 명확히 표시.

성능

- fal.ai 큐 등록 Promise.all(images/route.ts:73)은 유지하되 OpenRouter/fal 호출에 AbortSignal.timeout(30_000) 적용.
- studio/[id]/page.tsx의 중첩 쿼리를 image_jobs!inner 조인 1회 또는 뷰로 교체.
- landing-page.tsx(1,213줄) + hero-bg.mp4 2.8MB: 영상은 preload="none"/포스터 이미지, 섹션 지연 로딩.

개발 생산성 (빌드, 타입, 테스트, 중복 제거)

- 미사용 의존성 14개 제거(npm uninstall)로 설치 시간·감사 범위 축소. Stripe는 실제 구현 시점에 재추가.
- 죽은 코드 삭제: AuthProvider.tsx, projectStore.ts, camera.ts/useNativeCamera.ts(필요 시 UI 연결), fal.ts의 getFalResult/submitFalJobSync.
- supabase gen types typescript로 DB 타입 생성 -> types/index.ts 수동 타입과 동기화.
- 최소 테스트: deduct_credits RPC 동시성(pgTAP 또는 Vitest + 로컬 Supabase), zod 스키마, stripHtml, webhook 서명 검증.
- CLAUDE.md "현재 개발 환경 상태"와 proxy 패턴 설명을 실제 코드에 맞게 갱신.

운영/배포

- 환경변수 이름 통일(BROWSERLESS_API_KEY <-> BROWSERLESS_TOKEN), lib/env.ts에서 zod로 시작 시 검증, .env.example 추가.
- CSP 강화: unsafe-eval 제거(Next 16 프로덕션은 불필요), connect-src를 Supabase/fal/앱 도메인으로 한정.
- Capacitor: CAP_SERVER_URL 환경별 분리, cleartext/allowMixedContent 제거, 앱 아이콘/스플래시 리소스 생성 스크립트(@capacitor/assets) 추가.
- 월 크레딧 초기화(plan_reset_at) 크론(Supabase pg_cron 또는 Vercel Cron) 구현 - 현재 코드 없음.

5. 수정 작업 목록 및 권장 모델

#	작업	우선순위	예상 규모	권장 모델	선정 이유
1	크레딧 원자 차감 RPC(deduct_credits, refund_credits) 작성 + credit_logs 동일 트랜잭션 기록, /api/credits/use 교체	P0	M	Fable 5.1	결제/과금 로직, 동시성 버그, 실수 시 금전 피해
2	생성 라우트 6개에 "플랜·잔액 서버 조회 -> 차감 -> 생성 -> 실패 환불" 적용, images의 plan을 profiles.plan으로 교체, 클라이언트 selling_pages 직접 insert를 서버로 이동	P0	L	Fable 5.1	다중 라우트에 걸친 보안·과금 경계 재설계
3	image_prompts_service_role_update 정책 삭제 마이그레이션(003), fal webhook 서명/시크릿 검증 추가, final_url 기록 로직	P0	S	Fable 5.1	RLS·webhook 보안, 실수 비용 큼
4	Capacitor 배포 전략 확정 및 설정: server.url 환경 분리, cleartext/allowMixedContent 제거, webDir 정리, 앱 리소스 생성	P0	M	Opus 5.5	정의가 명확한 설정·연동 작업
5	crawl/url env 이름 수정, SSRF 방어(사설 IP·리다이렉트 차단), 크레딧 기록을 작업 1의 RPC로 교체; auth/callback의 next 화이트리스트·origin 고정	P1	S	Fable 5.1	입력 검증·리다이렉트 보안
6	StudioNewForm.tsx 분해(탭 7개 + 오케스트레이션 훅 + 상수), 단건/대량 생성 파이프라인 단일화	P1	L	Opus 5.5	명확한 리팩토링, 로직 보존 검증 필요
7	서버 데이터 조회 통합: requireUserWithProfile() + React.cache, studio/[id] 중첩 쿼리 조인으로 교체	P1	S	Opus 5.5	데이터 페칭 정리
8	대시보드 탭 목 데이터를 실제 조회로 교체(Projects=selling_pages, Billing=credit_logs), pages.tsx 파일 분리	P1	M	Opus 5.5	API 연동 + 중간 규모 리팩토링
9	미사용 의존성 14개 제거, 죽은 코드 삭제(AuthProvider, projectStore, fal.ts 미사용 함수, 루트 image*.png), .env.example, lib/env.ts zod 검증	P1	S	Sonnet 5.5	설정·정리 작업
10	외부 호출 타임아웃·재시도(fal 3회, PRD 9장) 및 JSON 파싱 실패 처리 표준화(lib/llm/openrouter.ts, lib/image/fal.ts)	P1	S	Opus 5.5	중간 난이도 안정성 개선
11	월 크레딧 초기화 크론(pg_cron) + 50/80/90% 소진 알림	P2	M	Fable 5.1	과금 관련 배치 로직
12	Stripe Checkout + webhook으로 추가 크레딧 구매(/api/credits/purchase) 구현	P2	L	Fable 5.1	결제 로직
13	CSP 강화, allowedDevOrigins 환경변수화, CLAUDE.md 상태/proxy 설명 갱신, 002 마이그레이션의 개인 이메일 시드 분리	P2	S	Sonnet 5.5	설정·문서 정리
14	플랫폼별 이미지 리사이즈(Sharp)·엑셀/ZIP 내보내기 라우트(PRD 4.5~4.7) 구현	P2	L	Opus 5.5	정의 명확한 기능 구현
15	자동화 캔버스 실행 엔진(연결 노드만 실행, bulk_register 크레딧 차감, 실행 로그)	P2	L	Fable 5.1	엔진 설계 + 과금 연계

6. 종합 평가

- 강점
  1. 전 API 라우트에 getUser() 인증 + zod 검증, any 0건, console.log 0건, Next 16 비동기 API 규칙 준수 등 코드 규율이 세 프로젝트 중 가장 높다.
  2. fal.ai 큐 -> webhook -> Supabase Realtime -> 뷰어 카드 갱신이라는 비동기 이미지 파이프라인이 실제로 연결되어 있고 RLS·가입 트리거를 포함한 마이그레이션이 저장소에 있다.
  3. OpenRouter 모델 역할 분리(분석/카피/비전/네이밍)와 프롬프트가 lib/llm에 모여 있어 모델 교체·튜닝이 쉽다.
- 약점

1. 크레딧 차감과 AI 생성이 별도 요청으로 분리되어 과금 우회가 가능하고, 차감 자체도 비원자적이다. 이미지 모델 선택 플랜도 클라이언트가 결정한다.
2. `USING(true)` UPDATE 정책과 무검증 webhook으로 이미지 데이터 무결성이 깨질 수 있다.
3. 2,919줄·2,515줄 거대 컴포넌트와 목 데이터 탭, 미사용 의존성 14개, 배포 불가 Capacitor 설정 등 "프로토타입 잔재"가 운영 코드에 섞여 있다.

- 모델별 비율(작업 15개 기준): Fable 5.1 47% (7건) / Opus 5.5 40% (6건) / Sonnet 5.5 13% (2건)
- 권장 진행 순서
  1. 1단계(P0): 작업 1->2->3으로 과금 경계와 RLS/webhook을 고친다. 이 전에는 유료 플랜 판매 금지. 병행으로 작업 4(Capacitor 배포 전략)를 결정한다.
  2. 2단계(P1): 작업 5~10 - 크롤링/콜백 보안, `StudioNewForm` 분해, 서버 조회 통합, 대시보드 실데이터, 의존성 정리, 타임아웃.
  3. 3단계(P2): 작업 11~15 - 크레딧 초기화 크론, Stripe, CSP/문서, 내보내기, 자동화 엔진.
- 주 담당 모델: **Fable 5.1**. 이 프로젝트는 UI와 코드 규율은 이미 양호하지만, 수익 모델의 핵심인 크레딧 차감이 비원자적이고 생성 라우트와 분리되어 있어 "돈이 새는 구조"를 트랜잭션 단위로 재설계하는 일이 최우선이다. RLS 정책 오류와 webhook 검증, 이후의 Stripe·크론·자동화 엔진까지 모두 실수 비용이 큰 결제·보안 영역이므로 Fable 5.1이 주도하고, 거대 컴포넌트 분해·대시보드 실데이터 연결·Capacitor 설정 같은 명확한 작업은 Opus 5.5, 의존성·문서·CSP 정리는 Sonnet 5.5로 분담한다.